

[Day 2] 종합 실습 · End2End 데이터 분석 프로젝트

데이터 분석을 위한 Python 이해 · 주제 : NYC 택시 운행 소요시간 예측 · Polars · Pandas · scikit-learn
광주 3반 4조 · 2026-07-21

0. 실습 요구사항 · 구현 체크

채점 영역	요구 사항	상태	구현 내용
데이터 준비 · 시각화	Pandas · Polars 양쪽 로딩 후 결과 비교	구현함	요일별 평균 소요를 두 엔진으로 집계해 완전 일치(True) 확인
	결측치 · 중복 · 이상치 처리	구현함	무효·이상치·중복 15.0만행 삭제, 인원 결측 70.3만 중앙값 대체
	기본 EDA (분포 · 관계)	구현함	describe 기술통계 · 상관행렬 · 소요시간 분포(치우침) 출력
	Seaborn 정적 차트 (제목·축라벨)	구현함	분포·시간 프로파일·거리×러시·상관·모델비교·중요도 6종
	Plotly 인터랙티브 차트	구현함	거리-소요 산점도·예측 vs 실제 2종 (HTML+PNG)
통계 분석 · ML	기술통계 (평균·표준편차·분위수)	구현함	describe 로 소요시간·거리 요약 출력
	변수 간 상관계수	구현함	상관행렬 산출, 거리 r=0.764 로 최강 신호 확인
	scipy ttest_ind t-test + p값 해석	구현함	Welch t-test 5건, p값으로 채택·기각 판정(거리 통제 포함)
	sklearn Pipeline 전처리 + 모델	구현함	FeatureBuilder→ColumnTransformer(타깃인코딩)→모델, 로그타깃 래퍼
	평가지표 출력 · 모델 저장	구현함	MAE·RMSE·R² (test) 출력, duration_model.joblib 저장
	여러 모델 비교 · 튜닝 · 앙상블	구현함	선형·RF·부스팅 비교 → HistGBM 튜닝 → 검증셋 블렌딩
자동화	분석 결과 report.md 자동 생성	구현함	report.py 로 데이터·가설·모델 결과를 report.md 로 자동 작성
완성도	주석 · docstring · 코드 품질	구현함	개조식 docstring, ruff 0, pytest 6 통과

1. 분석 개요

- 목표 — 택시 승차 시점에 알 수 있는 정보(거리·구역·시각·요일·인원) 만으로 **운행 소요시간(분)**을 예측
- 접근 — 다음 흐름으로 진행 : EDA → 피처 엔지니어링 → 모델링 → 튜닝 → 앙상블
- 도구 — 대용량은 Polars 로 빠르게 로딩·집계, Pandas 로 교차 확인, 모델링은 scikit-learn Pipeline

2. 시작 전에 — 핵심 통계 개념 5가지

회귀 예측 (Regression)

과거 데이터에서 규칙을 배워 연속된 숫자(여기선 소요시간, 분)를 맞히는 것. 잘 맞히는 정도를 R^2 (0~1, 1이면 완벽)로 표시.
쉽게 말하면 · 지난 시험의 공부시간·점수로 '이만큼 공부하면 몇 점' 을 가늠하는 것과 비슷.

상관계수 (기호 r)

두 값이 함께 움직이는지 보는 것. r 은 -1~+1 범위 (0에 가까우면 관계 약함, +1 이면 같이 오르고, -1 이면 반대로 움직임).
쉽게 말하면 · 이동 거리가 멀수록 소요시간도 길어짐 → '양의 상관'. 키와 몸무게도 비슷.

가설검정 (p값)

'이럴 것이다'라는 짐작(가설)이 우연인지 진짜인지 숫자로 판정. p값이 작을수록 (0.05 미만) '우연으로 보기 어렵다 = 진짜 차이'.
쉽게 말하면 · 동전을 10번 던져 9번 앞면이면 '이 동전 좀 이상한데?' 를 수치로 따지는 것.

타겟 인코딩 (Target Encoding)

구역 코드처럼 종류가 수백 개인 글자·번호를, 그 구역의 평균 소요시간 같은 숫자로 바꿔 모델이 읽게 하는 것.
쉽게 말하면 · 동네 이름 대신 '그 동네 평균 통행시간'을 적어 넣어 지역 특성을 숫자로 요약.

앙상블 (Ensemble)

성격이 다른 여러 모델의 예측을 섞어(평균·가중합) 한 모델의 실수를 다른 모델이 메우게 하는 기법.
쉽게 말하면 · 여러 사람의 추측을 평균 내면 한 사람보다 정답에 가까워지는 '집단 지성'과 같음.

3. 사용한 데이터

뉴욕시 택시위원회(TLC) 옐로 택시 운행 기록. 한 행이 택시 한 번의 운행이며, **승차 시점에 알 수 있는 값만** 설명변수로 골라 **소요시간**을 예측.

컬럼	의미	역할
trip_distance	이동 거리 (마일)	설명변수 (거리 — 소요시간 핵심 신호)
PULocationID	승차 구역 코드 (뉴욕 택시존)	설명변수 (출발지)
DOLocationID	하차 구역 코드	설명변수 (목적지)
pickup_hour	승차 시각 (0~23시)	설명변수 (시간대 혼잡)
pickup_weekday	승차 요일 (1=월 ~ 7=일)	설명변수 (요일 혼잡)
passenger_count	탑승 인원 수	설명변수 (보조)
duration_min	운행 소요시간 (분)	예측 대상 (target)

★ **누수(컨닝) 컬럼은 의도적으로 제외** — 아래 값들은 소요시간(시간)을 이미 담고 있어, feature 로 쓰면 '정답을 몰래 훑쳐보는' 데이터 누수가 됨. 실전에서 승차 순간엔 알 수 없는 값이기도 함 → 전부 제외하고 **승차 시점 정보만** 사용.

제외 컬럼	왜 누수인가
fare_amount · total_amount	요금·총액 — 미터기가 시간에 비례해 오르므로 소요시간을 이미 담음
tip_amount	팁 — 요금에 연동돼 간접적으로 소요시간 반영
tpcp_dropoff_datetime	하차 시각 — 탑승 시각과의 차이가 곧 정답(소요시간)
평균 속도(파생)	속도 = 거리 ÷ 소요시간 — 정답을 직접 나눠 만든 값

4. 데이터 심층 분석 (정제 · 결측 · 분포)

원본 4,090,836행을 정제 없이 8:2(train 80% · test 20%, seed 42)로 나눈 뒤, 학습 전 train 에 fit 하며 정제(무효·이상치·중복 삭제 + 인원 결측 대체).

아래는 train fold(3,272,668행) 기준이며, test 도 동일 규칙으로 처리(818,168 → 783,972행).

4-1. 정제 funnel — 무효·이상치·중복 단계별 제거

각 단계가 무엇을 왜 걸러내는지 행수와 함께. 규칙 정제는 fold 독립이라 train/test 사이 정보 누수 없음.

단계 (기준 · 근거)	제거	남은 행
원본 (분할된 train fold)	—	3,272,668
duration ≤ 0 — 하차≤승차, 무효 라벨(예측 불가)	41,491	3,231,177
duration > 180분 — 3시간 초과 과대 이상치	1,128	3,230,049
distance ≤ 0 — 이동 0, 무효	89,652	3,140,397
distance ≥ 100mi — 메트로 밖 과대 이상치	97	3,140,300
2026-05 이외 — 파일 선언 기간 밖 오류 날짜	9	3,140,291
완전중복 — 전 컬럼 동일 행	17,688	3,122,603
정제 완료	150,065	3,122,603

이상치를 IQR 대신 고정 범위컷으로 — 판단 근거

소요시간·거리는 오른쪽 통테일(공항 등 정상 장거리 존재) 분포라, IQR 상한 ($Q3+1.5\cdot IQR$)으로 자르면 정상 장거리 트립까지 이상치로 오인·삭제돼 모델이 짧은 트립에 편향됨. 그래서 물리·업무적 근거가 있는 도메인 범위(소요 1~180분 · 거리 0~100mi)로 컷 → 삭제량 4.6%로 작고 대부분 명백한 오류(19시간 트립·2008년 기록 등). test 도 같은 정의로 정제해 평가 일관성 유지.

4-2. 결측치 — 컬럼별 삭제·대치 판단

원본 20개 컬럼 중 결측치는 5개이며, 공교롭게 모두 같은 23.4% 행에서 함께 빈(수집 배치 누락으로 추정). 실제 대치는 모델에 쓰는 `passenger_count` 하나만 수행하지만, 안 쓰는 컬럼도 삭제할지·대치할지 처리 방침을 함께 명시 — 결측치는 반드시 방침을 정해 두는 것이 원칙(향후 피처로 쓸 경우 대비).

컬럼	피처	결측	삭제 · 대치 판단
<code>passenger_count</code>	사용	764,459	중앙값 대치(파이프라인, train fit) — 보조 피처라 대치 유리
<code>RatecodeID</code>	미사용	764,459	범주형 → '미상' 범주로 분리 대치
<code>store_and_fwd_flag</code>	미사용	764,459	범주 플래그 → '미상' 범주 대치
<code>congestion_surcharge</code>	미사용	764,459	요금 계열(시간 누수) → 피처 제외, 쓴다면 0 대치
<code>Airport_fee</code>	미사용	764,459	요금 계열(시간 누수) → 피처 제외, 쓴다면 0 대치

쓰는 피처인 거리·PU/DO 구역·시각·요일은 NULL 0건, 거리·소요시간의 '≤0'은 빈칸이 아니라 비정상 값이라 결측이 아닌 이상치(4-1)로 처리. 결국 실제 대치가 필요한 것은 `passenger_count` 하나지만, 위처럼 전 결측 컬럼의 처리 방침을 정해 둬.

왜 삭제가 아니라 중앙값 대치인가 — 판단 원칙

결측 처리는 '그 변수가 분석에서 하는 역할'에 따라 같림.

① 인원은 부차 설명변수(피처 중요도 0.001로 거의 영향 없음)이고, 같은 행의 거리·구역·시각은 소요시간 예측에 유효. 인원 하나 비었다고 22.5%(약 70만 행)를 통째로 버리면 엄청난 학습 데이터만 크게 손해 → 삭제는 낭비.

② 그래서 빈 인원(및 0·음수 같은 비정상 9,690건)만 중앙값으로 통일해 채워 행을 살림. 대치는 train에만 fit(SimpleImputer)해 test 로의 누수 차단. 평균이 아니라 중앙값을 쓰는 이유 = 9명 같은 극단값에 안 휘둘러 안정적.

③ 반대로 그 변수 자체가 분석의 핵심(예: 인원을 직접 비교하는 가설)이라면, 없는 값을 지어내는 대치가 결과를 왜곡하므로 그때는 삭제가 옳음 — 대치와 삭제는 상황에 따라 달리 적용하는 것이 일관된 원칙.

④ 실제로 같은 택시 데이터라도 인원·요금을 직접 집계·비교하는 EDA에서는 인원이 비었거나 1명 미만인 행을 비정상 거래로 보고 삭제하는 편이 맞음 — 목적이 '집계·비교의 신뢰도'라 지어낸 값이 통계를 흔들기 때문. 이 프로젝트는 목적이 소요시간 예측이고 인원은 보조 피처라, 같은 결측이라도 대치가 유리. 목적이 다르면 처리도 달라지는 것이 모순이 아니라 원칙.

4-3. 예측 대상(소요시간) 분포 — 오른쪽 치우침 → 로그 변환

지표	평균	중앙	표준편차	p95	최대	치우침(skew)
소요시간 (원본, 분)	18.80	14.67	15.44	48.82	179.98	2.48
소요시간 (log1p 변환)	—	—	—	—	—	-0.03

왜 로그로 학습하나 — 소요시간은 짧은 트립에 몰리고 긴 트립이 꼬리로 늘어져 오른쪽으로 크게 치우침(skew 2.48). 이대로 학습하면 모델이 긴 트립에 과하게 끌려감. log1p 를 씌우면 분포가 좌우 대칭 종 모양에 가까워짐 (skew -0.03) → 안정적. 그래서 로그 소요시간을 학습하고, 예측 뒤 지수로 되돌려 분(min) 단위로 환산.

skew(치우침) = 분포가 한쪽으로 쏠린 정도. 0이면 좌우 대칭, 클수록 한쪽 꼬리가 김.

5. 종합실습 요구사항과 해결

① 데이터 준비

요구 · 선택 데이터셋을 Pandas·Polars 양쪽으로 로딩·비교, 결측·중복 처리, 기본 EDA

방법 · 원본 409만 건을 무정제로 train/test 분할 후, train 에 fit 하며 하류 정제(무효·이상치·중복 삭제 + 인원 결측 중앙값 대체) → train 312만·test 78만. Pandas·Polars 양쪽 집계·describe 기술통계 출력

결과 · Pandas·Polars 집계 완전 일치(True) 확인 → 대용량도 안정 처리

② 시각화

요구 · Seaborn 정적 차트 + Plotly 인터랙티브 차트 각 1개 이상 (제목·축 레이블)

방법 · Seaborn 정적 6종(분포·시간대·거리구간·상관·모델비교·중요도) + Plotly 2종(거리-소요 산점도·예측-실제 산점도), 모두 제목·축라벨 포함

결과 · 정적·인터랙티브 요건 충족, 총 차트 8종 생성

③ 통계 분석

요구 · 기술통계(평균·표준편차·분위수)·상관계수·scipy ttest_ind t-test + p값 해석

방법 · describe·상관행렬 출력 + scipy.stats.ttest_ind 로 가설 5종 검정, 각 결과의 p값을 채택·기각으로 해석

결과 · 5종 전부 채택, p값 근거로 러시·거리·심야 효과 확인

④ ML Pipeline

요구 · sklearn Pipeline 으로 전처리+모델 구성, 여러 모델 비교·튜닝·양상블, joblib 저장

방법 · FeatureBuilder→ColumnTransformer→모델을 Pipeline 으로 묶어 소요시간 회귀, 3개 모델 비교 + HistGBM 튜닝 + 양상블, R^2 ·MAE 출력 후 joblib 저장

결과 · 양상블 최고(R^2 0.794·MAE 4.05분), duration_model.joblib 저장 확인

⑤ 자동화

요구 · 분석 결과를 report.md 로 자동 생성

방법 · 모든 단계 결과(데이터·가설·모델 비교·산출물)를 모아 report.md 를 코드로 자동 작성

결과 · 실행 한 번으로 리포트 재생성 — 재현성 확보

6. 실행 결과 — 파이프라인 전체 출력

위 '결과'가 실제로 나온 실행 로그. 데이터 준비·기술통계(describe)·상관·가설 5종 검정·모델 튜닝·모델별 성능 (R^2 ·MAE)·피쳐 중요도·리포트 자동생성까지 코드 실행 출력 그대로 수록.

데이터 준비 · 기술통계(describe) · 상관계수 · 가설 5종 검정

```
$ python -m trip_duration.main - 데이터 · 기술통계 · 상관 · 가설
```

```
===== NYC 택시 운행 소요시간 예측 =====
```

```
[1] 데이터 준비 (원본 무정제 분할 → 하루 정제)
```

```
원본 총 4,090,836행
```

```
train 정제 funnel:
```

```
원본 (분할된 fold)                → 3,272,668
duration ≤ 0 (하차≤승차, 무효 라벨)  -41,491 → 3,231,177
duration > 180분 (과대 이상치)       -1,128 → 3,230,049
distance ≤ 0 (무효)                 -89,652 → 3,140,397
distance ≥ 100mi (과대 이상치)       -97 → 3,140,300
2026-05 이외 (오류 날짜)            -9 → 3,140,291
완전중복 제거                       -17,688 → 3,122,603
```

```
train 3,122,603 / test 783,972 · 인원 결측 대치 703,059(train)
```

```
Pandas·Polars 일치 : True
```

```
[2] 기술통계 (소요시간·거리)
```

	duration_min	trip_distance
count	3122603.00	3122603.00
mean	18.80	3.52
std	15.44	4.25
min	0.02	0.01
25%	8.87	1.10
50%	14.67	1.93
75%	23.43	3.94
95%	48.82	12.62
max	179.98	99.66

```
상관(소요시간 기준):
```

```
trip_distance    0.764
passenger_count  0.003
pickup_hour      -0.008
pickup_weekday   -0.047
```

```
[3] 가설 검정
```

```
H1 러시아워엔 소요시간이 더 긴가 → 채택 (러시 19.1분 vs 그외 18.6분, p=3.8e-115)
```

```
H2 거리가 길수록 소요시간이 긴가 → 채택 (r = 0.764)
```

```
H3 주말이 평일보다 소요시간이 다른가 → 채택 (주말 17.3분 vs 평일 19.5분, p=0.0e+00)
```

```
H4 심야(0~5시)엔 소요시간이 짧은가 → 채택 (심야 15.5분 vs 그외 19.1분, p=0.0e+00)
```

```
H5 거리(3~5mi)를 통제해도 러시가 더 긴가 → 채택 (러시 24.5분 vs 그외 22.3분, p=0.0e+00)
```

HistGBM 튜닝 · 모델별 성능 비교(선형·랜덤포레스트·부스팅·앙상블)

HistGBM 무작위탐색 튜닝 · 모델별 성능 비교(MAE·RMSE·R²)

[4] HistGBM 튜닝 (부분표본 400,000, cv-MAE 4.2241분)

```
{'min_samples_leaf': 50, 'max_leaf_nodes': 127, 'max_iter': 300, 'max_depth': None, 'learning_rate': 0.1, 'l2_regularization': 0.1}
```

[5] 모델 비교 (test)

- 선형회귀 (Ridge) MAE 5.403 / RMSE 8.980 / R2 0.661
- 랜덤포레스트 MAE 4.205 / RMSE 7.180 / R2 0.783
- 히스토그램 부스팅 (튜닝) MAE 4.048 / RMSE 7.005 / R2 0.793
- 앙상블 (블렌딩) MAE 4.046 / RMSE 6.989 / R2 0.794 *최고

앙상블 가중치 : {'선형회귀 (Ridge)': 0.0, '랜덤포레스트': 0.2347, '히스토그램 부스팅 (튜닝)': 0.7653}

피쳐 중요도 · 시각화 · 리포트/모델 자동 저장

피쳐 중요도 · 시각화 8종 · report.md · 모델(joblib) 저장

[6] 피쳐 중요도 상위 : [('trip_distance', np.float64(0.495)), ('log_distance', np.float64(0.347)), ('DOLocationID', np.float64(0.033)), ('PULocationID', np.float64(0.031)), ('pickup_hour', np.float64(0.026))]

[7] 시각화 : 01_target_dist.png, 02_hour_profile.png, 03_dist_rush.png, 04_corr.png, 05_model_mae.png, 06_importance.png, 07_scatter.html, 08_pred_actual.html

[8] 리포트 : report.md · 모델 : duration_model.joblib

완료

7. 분석 질문과 답

Q1. 소요시간은 무엇으로 예측되는가

답 · 거리가 압도적. 구역·시각이 보조 신호.

이동 거리와 소요시간의 상관인 $r=0.764$ 로 가장 강함. 승하차 구역·승차 시각도 혼잡 정보를 담아 예측을 돕지만, 거리에 비하면 기여가 훨씬 작음.

Q2. 같은 거리인데 러시아워면 더 오래 걸리는가

답 · 그렇다. 거리를 맞춰도 러시아워가 유의하게 더 김.

거리 3~5mi 로 조건을 통제해 비교했더니 러시 24.5분 vs 그외 22.3분($p \approx 0$). 거리를 걷어낸 뒤에도 남는 차이라, '시각 (혼잡) 자체'가 소요시간을 늘리는 원인에 가까움.

Q3. 어떤 모델이 가장 정확한가

답 · 앙상블(블렌딩). MAE 약 4.05분.

부스팅·랜덤포레스트를 가중 블렌딩한 앙상블이 test MAE 4.046분· R^2 0.794 로 근소 1위. 튜닝한 히스토그램 부스팅 (4.048)과 사실상 동률 — 부스팅 단독으로도 충분.

8. 가설 5종 검증

각 직관을 귀무가설 H_0 (차이·관계 없음)와 대립가설 H_1 (차이·관계 있음)로 세우고,
유의수준 5%($\alpha=0.05$)에서 Welch's t-test(분산이 달라도 되는 평균 비교)로 판정.
5개 모두 채택 — 거리·시각(러시·심야)·요일이 소요시간을 좌우함을 확인.

번호	세운 가설	확인 방법	결과	판정	상관 / 인과
H1	러시아워엔 소요시간이 더 길까	Welch t-test (러시 vs 그외)	러시 19.1분 vs 그외 18.6분 $p = 3.8e-115$	채택	인과 개연성 (출퇴근 혼잡)
H2	거리가 멀수록 소요시간이 길까	피어슨 상관 (거리 vs 소요)	$r = 0.764$ (강한 양의 관계)	채택	인과에 가까움 (물리적 거리)
H3	주말과 평일의 소요시간이 다를까	Welch t-test (주말 vs 평일)	주말 17.3분 vs 평일 19.5분 $p \approx 0$	채택	연관 (통근 수요 개입)
H4	심야(0~5시)엔 소요시간이 짧을까	Welch t-test (심야 vs 그외)	심야 15.5분 vs 그외 19.1분 $p \approx 0$	채택	인과 개연성 (도로 한산)
H5	거리(3~5mi) 통제해도 러시가 더 길까	Welch t-test (동일 거리대)	러시 24.5분 vs 그외 22.3분 $p \approx 0$	채택	★ 인과 개연성 (거리 통제 후 유지)

용어 풀이

- r (상관계수) — 두 값이 함께 움직이는 정도(-1~+1). 0이면 무관, +1이면 완전히 같이 오름. 0.76처럼 크면 '강한 관계'.
- p (p값) — '정말 차이가 없다고 가정했을 때, 지금 관측된 만큼의 차이가 우연히 나올 확률'. **0.05 미만이면 우연이 아니라 유의미한 차이로 봄.**
- MAE — 예측이 실제와 평균 얼마나 벗어나는지(분 단위, 작을수록 정확).

대표본에선 p 가 거의 다 유의 → 효과크기를 함께 봐야 — 표본이 수백만 건이면 아주 작은 차이도 '우연이라기엔 너무 일관된' 것으로 잡혀 p 값이 0에 수렴함 (H1·H3·H4·H5 의 $p \approx 0$ 도 그런 맥락). 그래서 p 값만 보지 말고 **실제 차이의 크기** 를 같이 봐야 함
— 예: H1 러시 차이 0.4분은 통계적으로 유의하나 체감은 작고, H5 러시 차이 2.2분은 크기도 의미 있음.

9. 피쳐 엔지니어링 — 왜 이렇게 만들었나

원본 컬럼만으로는 부족 → 데이터가 보여준 관계를 근거로 파생 피쳐를 설계.
각 피쳐는 '왜 필요한가'와 '데이터 근거'를 함께 정리.

log1p(거리) — 로그 변환

왜 · 거리와 소요시간의 관계가 원본에선 살짝 휘지만, 로그 축에선 더 곧은 직선.

근거 · 거리-소요 상관이 $r\ 0.764 \rightarrow 0.775$ 로 개선 → 선형 모델도 더 잘 잡음.

순환 시각 (sin·cos)

왜 · 23시와 0시는 숫자로는 멀지만 실제로는 바로 이웃. 시각을 원(circle)에 얹어 sin·cos 두 값으로 표현하면 자정 경계가 자연스럽게 이어짐.

근거 · 23시↔0시가 인접한 값이 되어 밤샘 시간대 패턴을 끊김 없이 학습.

is_rush · is_night · is_weekend

왜 · 러시아워·심야·주말 여부를 0/1 깃발로 만들어 혼잡 신호를 명시적으로 전달.

근거 · 가설 H1·H4·H3 에서 유의했던 시간대·요일 효과를 모델이 바로 활용.

log거리 × 러시 (상호작용)

왜 · 러시의 지연 효과가 거리에 비례해 커진다는 가정을 곱셈 항으로 표현.

근거 · 거리 5~10mi 구간에서 러시 소요가 약 +14% → 상호작용이 실제로 존재.

타깃 인코딩 (PU/DO 약 260종)

왜 · 승하차 구역은 종류가 260여 개(고카디널리티) → 원-핫이면 열이 폭발. 대신 구역별 평균 소요를 수치로 넣되, 내부 교차적합으로 정답 누수 차단.

근거 · 260여 구역을 열 2개로 압축하면서 지역 혼잡 특성을 숫자로 보존.

10. 모델링 — 전처리 · 모델 · 튜닝 · 앙상블

10-1. 전처리 — 왜 이렇게 준비했나

- **중앙값 대체** — 빈 인원 값을 중앙값으로 채움. 극단값(9명)에 견고해 트리 모델에 안정적.
- **표준화 (StandardScaler)** — 거리·시각 등 단위·크기가 제각각인 수치를 평균 0·폭 1로 맞춰 공정하게 비교 (특히 선형 모델에서 중요).
- **타깃 인코딩** — 종류 260여 개인 승하차 구역을 원-핫 대신 구역별 평균 소요로 수치화, 내부 교차적합으로 정답 누수 차단.
- 이 셋을 **ColumnTransformer**로 묶고 모델과 한 흐름(Pipeline)으로 연결 → 새 데이터에도 같은 처리가 자동 적용.

10-2. 쓴 모델 3가지 — 각자 뒤에 강한가

- **Ridge (선형 기준선)** — 피처에 가중치를 곱해 더하는 직선식. 빠르고 해석이 쉬워 **기준선(baseline)** 역할. 곡선 관계는 덜 잡음.
- **RandomForest (랜덤포레스트)** — 서로 다른 결정 트리 여러 그루의 예측을 평균. 비선형·상호작용을 잘 잡고 이상치에 강해 **표 데이터에 튼튼**.
- **HistGradientBoosting (히스토그램 부스팅)** — 앞 트리의 오차를 다음 트리가 메우도록 순서대로 쌓는 부스팅. 정확도가 높고 히스토그램 가속으로 **대용량에 빠름**.

10-3. 튜닝 — HistGBM 무작위탐색

부분표본 40만 건으로 무작위탐색(RandomizedSearchCV) 수행, 교차검증 cv-MAE 4.22분. 최적 하이퍼파라미터 :
learning_rate 0.1 · max_leaf_nodes 127 · max_iter 300 · max_depth None · l2_regularization 0.1 · min_samples_leaf 50.

10-4. 앙상블 — 검증셋 비음수 최소제곱 가중치

검증셋에서 비음수 최소제곱(NNLS)로 세 모델의 최적 가중치를 구함 → Ridge 0.0 · RandomForest 0.235 · HistGBM 0.765.
부스팅이 가중치를 지배. test 셋에 이 가중치로 블렌딩.

10-5. 평가지표 읽는 법 — MAE · RMSE · R²

아래 표의 세 숫자가 무엇인지 · 크면/작으면 좋은지 · 어떻게·왜 계산하는지 먼저 정리 (모두 test 셋 기준).

지표	뜻 (쉽게)	방향	계산 · 왜 그렇게
MAE (평균 절대 오차)	예측이 실제에서 평균 몇 분 빗나갔나	낮을수록 좋음	계산 · 예측-실제 (부호 무시)의 평균 왜 · '평균 몇 분 틀림'을 그대로 읽고, 튀는 값에 덜 휘둘림
RMSE (제곱근 평균제곱오차)	큰 실수에 더 벌점을 준 평균 오차(분)	낮을수록 좋음	계산 · (예측-실제) ² 의 평균 → √ 왜 · 큰 오차를 크게 벌함(가끔 크게 틀리는 걸 싫을 때). 항상 RMSE ≥ MAE
R ² (결정계수)	소요시간의 들쭉날쭉함을 모델이 몇 % 설명하나	높을수록 좋음 (1=완벽)	계산 · 1 - (모델 오차 ² 합 ÷ 평균 찍기 오차 ² 합) 왜 · '평균 찍기' 대비 나은 비율, 단위 없어 비교 쉬움(0=평균·1=완벽)

이번 결과를 말로 풀면 — 최고 모델(앙상블) MAE 4.05분 = 예측이 실제 소요시간에서 평균 약 4분 벗어남. RMSE 6.99분 = 큰 실수에 벌점을 준 오차라 MAE보다 큼(가끔 크게 틀리는 장거리·비침두가 섞였다는 뜻). R² 0.79 = 소요시간이 왜 제각각인지의 약 79%를 거리·시각·구역으로 설명 — 나머지 21%는 사고·신호·개별 운전 습관처럼 데이터에 없는 요인.

10-6. 모델별 성능 비교 (test)

모델	MAE (↓ 좋음)	RMSE (↓ 좋음)	R ² (↑ 좋음)
선형회귀 (Ridge)	5.403	8.980	0.661
랜덤포레스트	4.205	7.180	0.783
히스토그램 부스팅 (튜닝)	4.048	7.005	0.793
앙상블 (블렌딩) ★ 최고	4.046	6.989	0.794

피쳐 중요도 (RandomForest 기준)

피쳐	중요도
trip_distance	0.495
log_distance	0.347
DOLocationID	0.033
PULocationID	0.031
pickup_hour	0.026

해석 — 거리 계열(trip_distance·log_distance)이 **84%**를 차지해 압도적. 모델은 **양상불이 근소 1위**(MAE 4.046)이나 튜닝 부스팅 (4.048)과 사실상 동률 — HistGBM 이 가중치 **0.77** 로 지배해 블렌딩 이득이 미미. '양상불이 항상 크게 낮지는 않음'을 정직하게 확인.

R^2 = 실제를 얼마나 잘 맞히는지(1=완벽) · **MAE** = 평균 오차(분) · **RMSE** = 큰 오차에 더 민감한 평균 오차(분).

11. 겉보기 관계 ≠ 진짜 원인 (상관 vs 인과)

상관 = 두 값이 같이 움직이는 것 / 인과 = 한쪽이 다른 쪽을 실제로 '일으키는' 것. 둘은 다름 — 같이 움직인다고 서로 원인인 것은 아님.

11-1. 이번 분석의 핵심 — H5 (거리 통제 후에도 러시가 김)

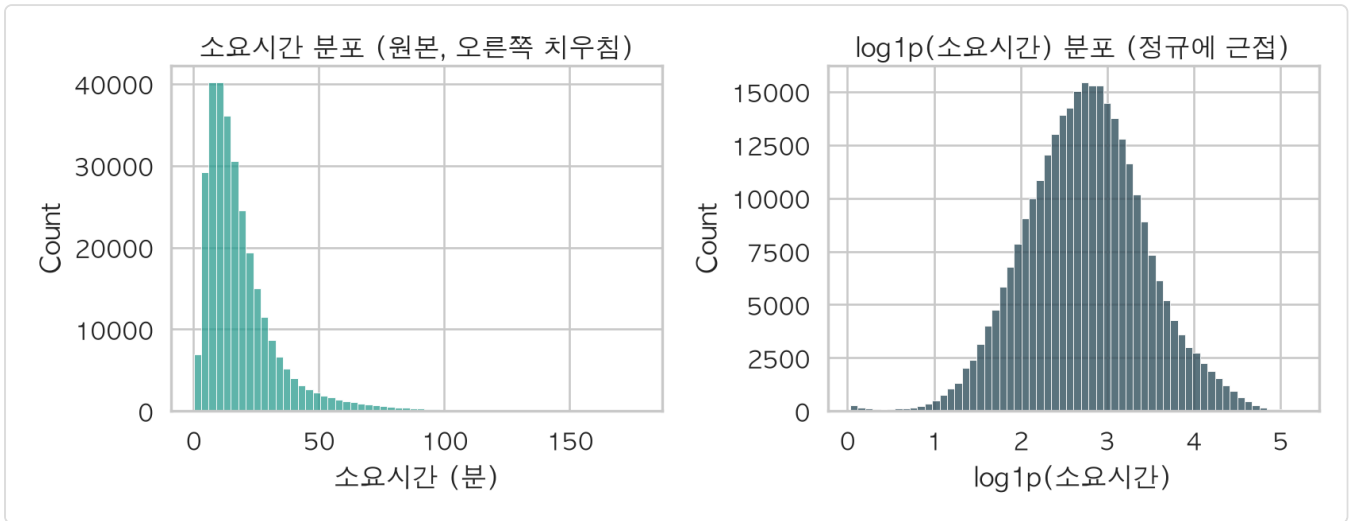
- 단순히 '러시엔 소요시간이 길다'만 보면, 사실은 **러시 때 장거리 트립이 많아서** 길어 보이는 착시일 수 있음 (거리가 숨은 원인)
- 그래서 **거리 3~5mi 로 조건을 맞춘 뒤(통제)** 러시/그외를 비교 → 러시 24.5분 vs 그외 22.3분($p \approx 0$). 거리를 걷어낸 뒤에도 **차이가 남음**
- 즉 '시각(혼잡) 자체'가 소요시간을 늘리는 원인에 가깝다고 볼 근거 → 단순 상관을 넘어 **인과 개연성**을 확인한 사례

11-2. 그래도 남는 한계 (관측연구의 정직한 선)

- 거리는 통제했지만 **날씨·사고·도로공사** 같은 미관측 요인은 통제 못 함 — 이들이 러시 시간대와 겹치면 러시 효과에 섞여 들어갈 수 있음
- 따라서 '시각이 소요시간에 영향을 준다'는 **강한 개연성**까지가 관측 데이터가 허락하는 선. 완전한 인과 확정에는 추가 통제·외부 데이터가 필요 — 이 한계를 숨기지 않고 그대로 명시

12. 시각화와 해석

① 소요시간 분포 (원본 vs 로그)



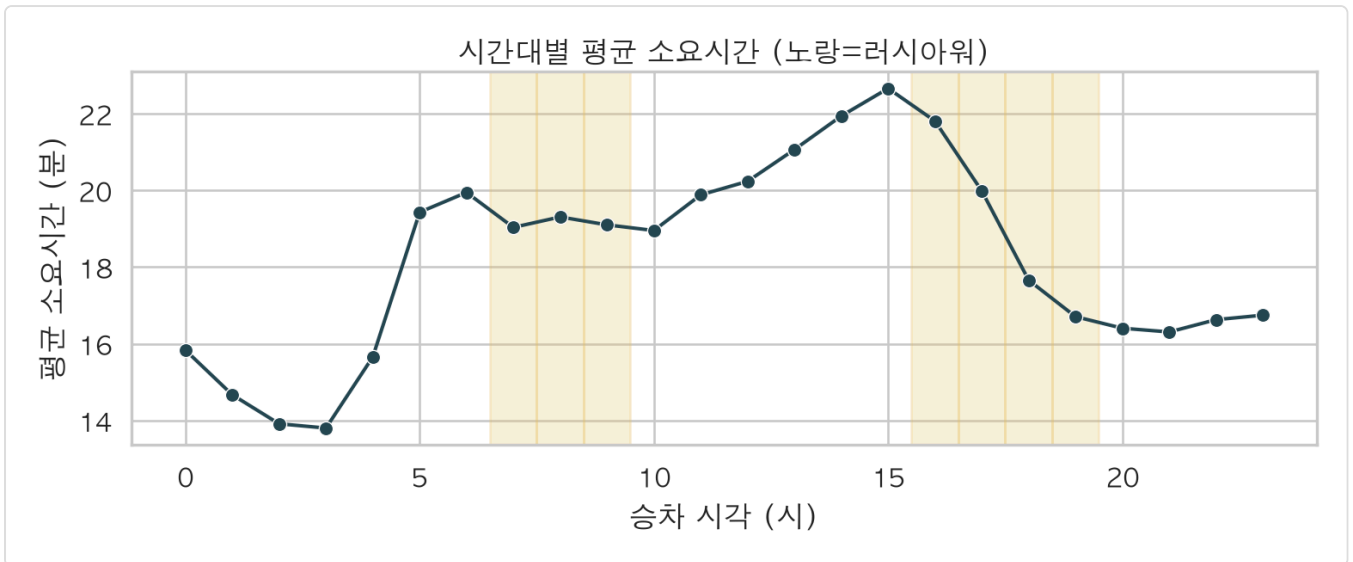
왜 이 그래프 · 예측 대상인 소요시간이 어떤 모양으로 퍼졌는지 먼저 봐야 학습 방식을 정할 수 있음. 원본과 로그 변환본을 나란히 두어 치우침이 얼마나 퍼지는지 비교.

무엇을 보나 · 소요시간의 분포 모양 · 오른쪽 꼬리(긴 트립) · 로그 변환의 효과.

읽어보면 · 원본은 짧은 트립에 몰려 오른쪽으로 길게 늘어짐(skew 2.48). 로그를 씌우면 좌우 대칭 종 모양에 가까워짐(skew -0.03).

다음으로 · 치우친 타겟은 로그로 학습해야 안정적 → 이후 모델은 log 소요시간을 예측.

② 시간대별 평균 소요시간 (러시 음영)



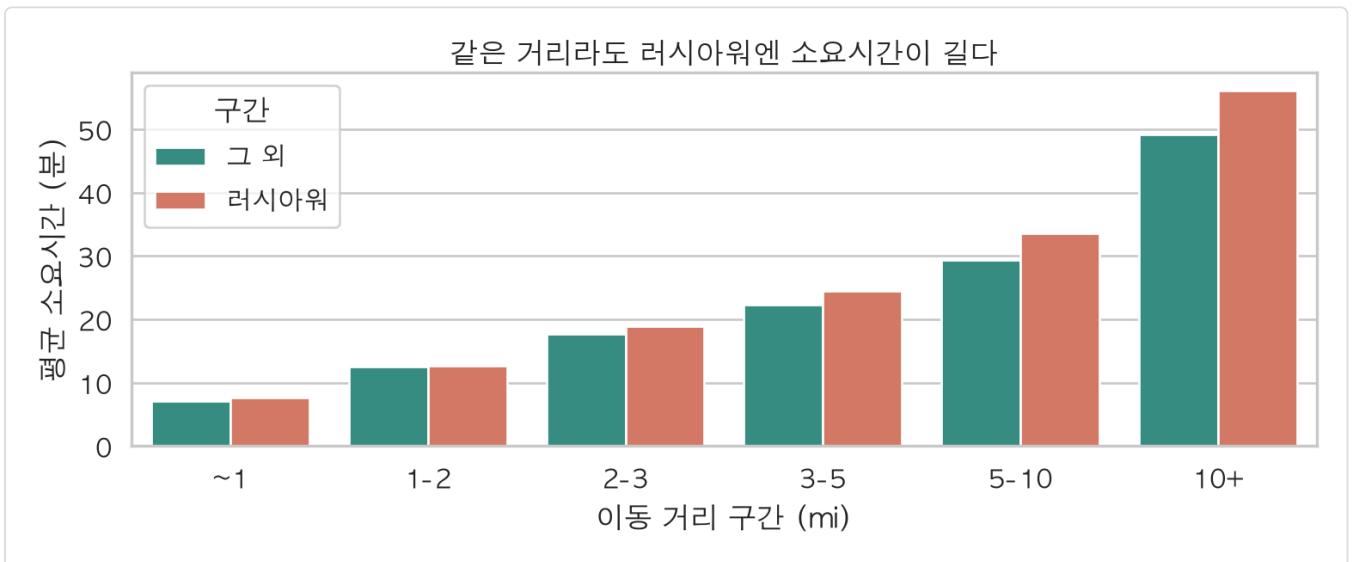
왜 이 그래프 · '몇 시에 더 오래 걸리나'를 보려면 24시간 프로파일이 적합. 러시아워 구간을 음영으로 덧대 혼잡 시간대를 눈에 띄게 표시.

무엇을 보나 · 시각에 따른 평균 소요시간의 오르내림과 러시아워 구간.

읽어보면 · 출퇴근 러시 구간에서 평균 소요가 봉우리를 이루고, 심야에 가장 낮은 골을 만들.

다음으로 · 시각이 소요시간을 흔든다는 신호 → 거리를 통제해도 러시가 남는지 다음 차트로.

③ 거리구간 × 러시 막대 (상호작용)



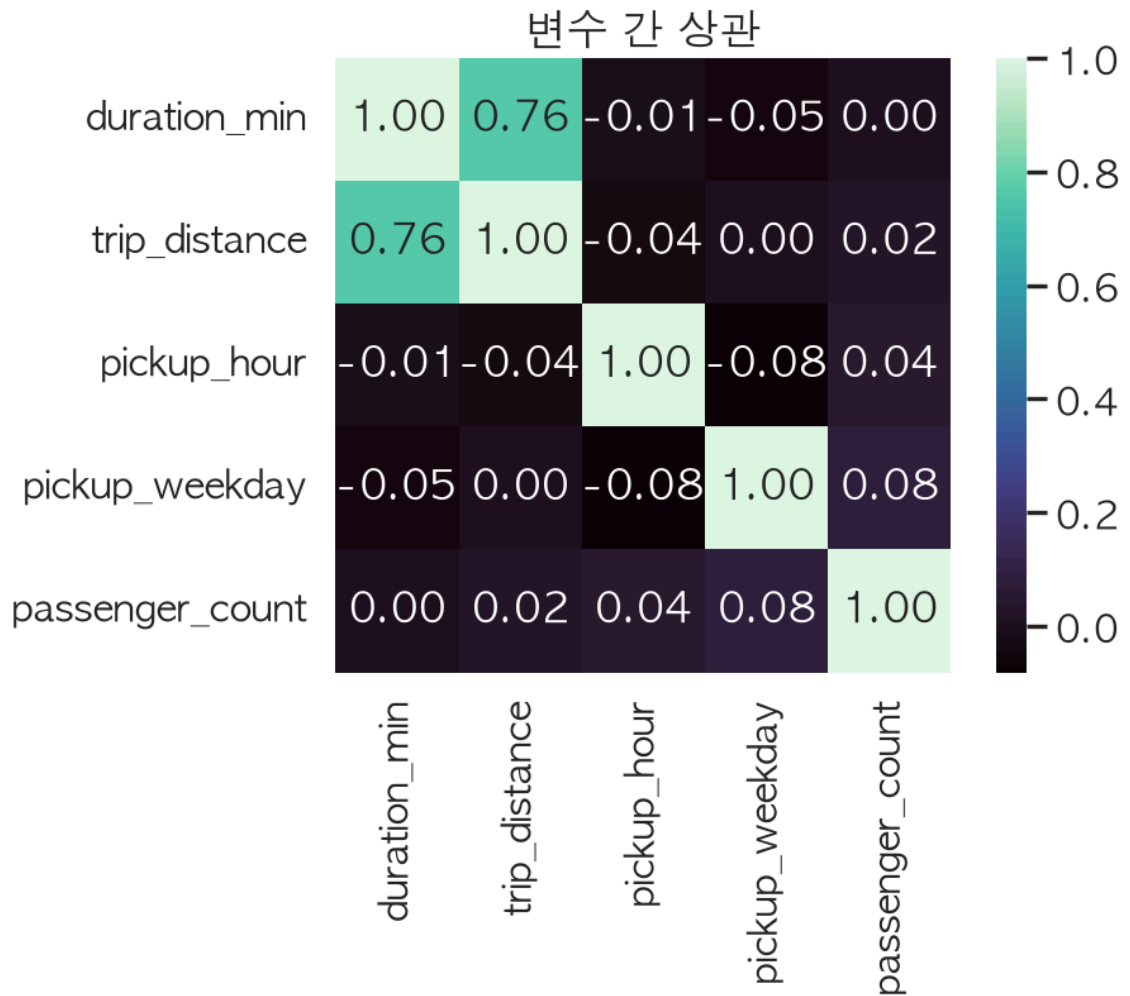
왜 이 그래프 · 러시 효과가 거리에 따라 달라지는지(상호작용) 보려면 거리 구간별로 러시/그외를 나란히 세운 막대가 직관적.

무엇을 보나 · 같은 거리 구간 안에서 러시와 그외의 소요시간 차이.

읽어보면 · 모든 거리 구간에서 러시 막대가 더 높고, 거리가 늘수록 그 격차도 벌어짐 → 러시 지연이 거리에 비례해 커짐.

다음으로 · 거리×러시 상호작용이 실재함을 확인 → 피쳐로 공급 함을 추가한 근거.

④ 상관 히트맵



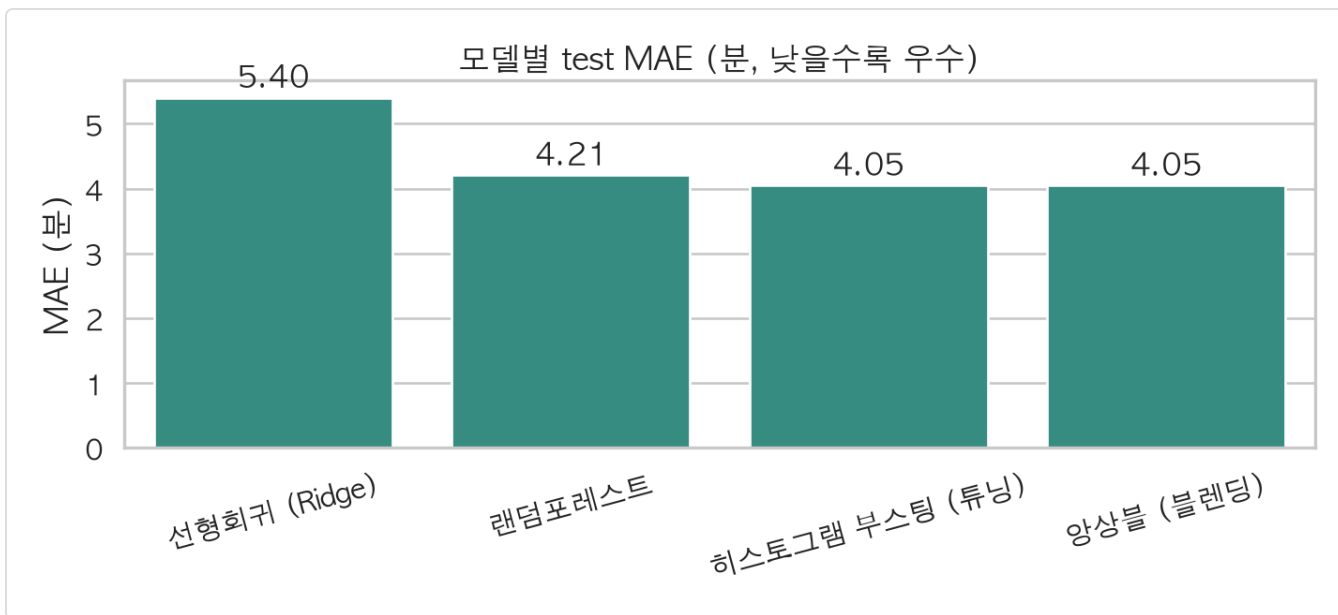
왜 이 그래프 · 여러 변수 쌍의 상관을 숫자표로 나열하면 눈에 안 들어옴 → 색으로 강약을 한눈에 보여주는 히트맵으로 종합.

무엇을 보나 · 소요시간과 각 변수의 상관 방향·세기, 서로 겹치는 변수(다중공선성).

읽어보면 · 거리-소요가 가장 진한 양(0.76), 시각·요일은 열음. log_distance↔trip_distance 처럼 강하게 겹치는 쌍도 보임.

다음으로 · 거리가 핵심 신호임을 재확인 → 이 관계를 학습시킨 것이 아래 모델.

⑤ 모델별 MAE 비교



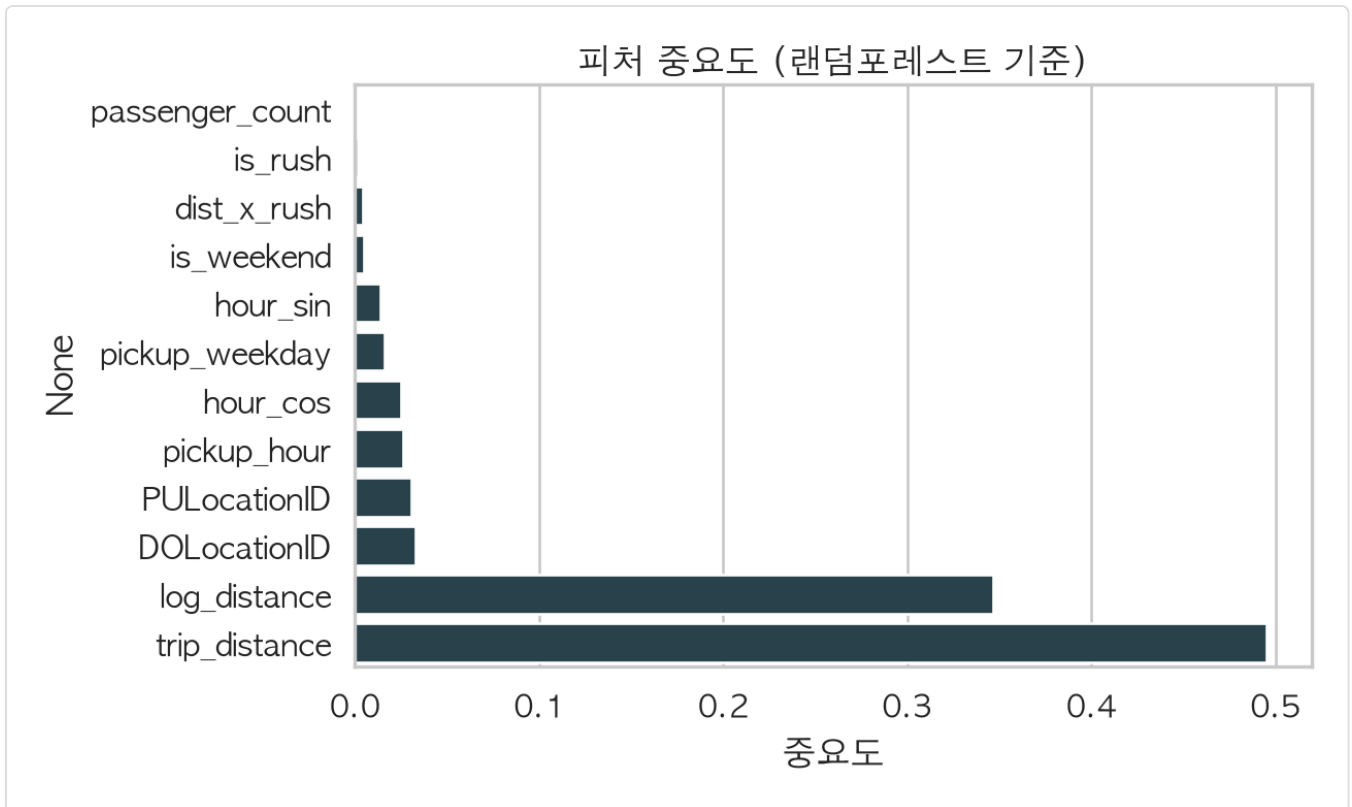
왜 이 그래프 · 여러 모델의 정확도를 한눈에 줄 세우려면 MAE(작을수록 좋음) 막대 비교가 명확.

무엇을 보나 · 선형·랜덤포레스트·부스팅·앙상블의 평균 절대오차.

읽어보면 · 선형(5.40) → 랜덤포레스트(4.21) → 부스팅(4.05)로 낮아지고, 앙상블(4.05)은 부스팅과 사실상 같음.

다음으로 · 부스팅·앙상블이 최고임을 확인 → 그 모델이 무엇을 중요하게 보는지 다음 차트로.

⑥ 피처 중요도 (RandomForest)



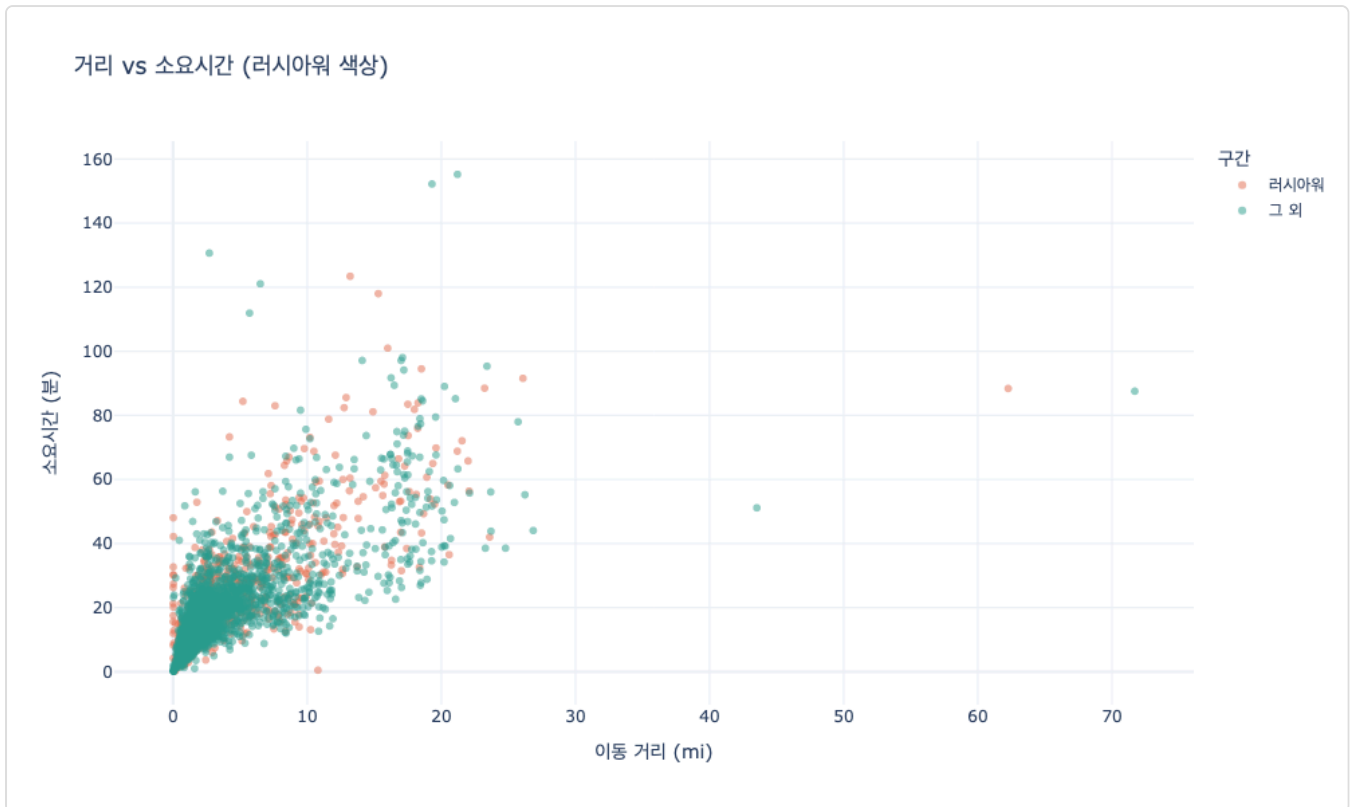
왜 이 그래프 · 모델이 어떤 값에 크게 의존하는지 알아야 결과를 해석할 수 있음 → 중요도 막대로 순위화.

무엇을 보나 · 각 피처가 소요시간 예측에 기여한 상대적 크기.

읽어보면 · trip_distance(0.495)·log_distance(0.347)로 거리 계열이 84%를 차지, 구역·시각은 각 3% 안팎.

다음으로 · 거리가 압도적 지배 신호임을 수치로 확정 → 거리·시각을 실제 점으로 확인하러.

⑦ 거리 vs 소요시간 산점도 (러시 색, 인터랙티브)



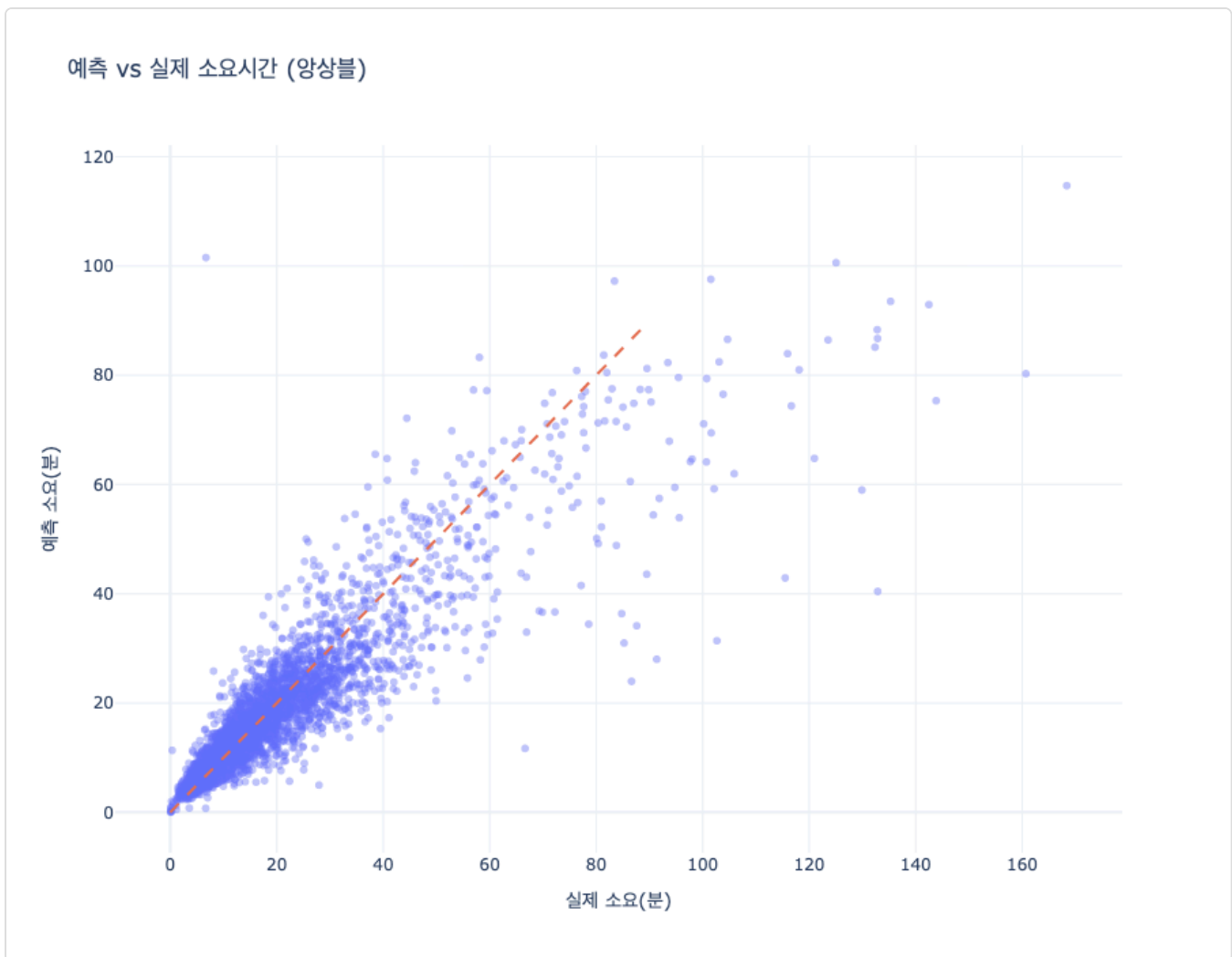
왜 이 그래프 · 거리-소요 관계의 '모양과 흩어짐'까지 보려면 개별 트립을 점으로 뿌리는 산점도가 적합. 러시 여부를 색으로 구분해 혼잡 효과를 겹쳐 봄.

무엇을 보나 · 거리에 따른 소요시간의 증가 추세와 러시/그외 점의 분포.

읽어보면 · 점들이 우상향 띠를 이루며 거리와 함께 소요가 늘. 같은 거리대에서 러시 색 점들이 위쪽(더 오래)에 더 몰림.

다음으로 · 거리·러시 효과를 점으로 재확인 → 마지막으로 모델이 실재를 얼마나 맞히는지.

⑧ 예측 vs 실제 산점도 (대각선)



왜 이 그래프 · 모델의 정확도를 직관적으로 보려면 예측값과 실제값을 대각선 기준으로 흘뿌리는 것이 가장 명확(대각선에 붙을수록 정확).

무엇을 보나 · 예측이 실제와 얼마나 일치하는지, 어디서 크게 빗나가는지.

읽어보면 · 점들이 대각선을 따라 촘촘히 모여 있어 예측이 대체로 정확. 아주 긴 트립에서 약간 과소예측하는 꼬리가 보임.

다음으로 · 평균 4분 안팎 오차(R^2 0.79)로 소요시간을 예측함을 시각으로 확인.

13. 데이터가 말해주는 것 (핵심 정리)

소요시간은 '거리'가 지배

이동 거리와 소요시간의 상관관계 $r=0.764$ 로 가장 강하고, 피쳐 중요도에서도 거리 계열이 84%를 차지 — 무엇보다 '얼마나 멀리 가느냐'가 시간을 결정.

같은 거리라도 러시엔 더 오래 (혼잡)

거리 3~5mi 로 조건을 맞춰도 러시 24.5분 vs 그외 22.3분. 거리를 통제된 뒤에도 남는 차이라, 시각(혼잡) 자체가 소요시간을 늘리는 원인에 가까움.

심야는 빠름

0~5시 심야는 15.5분으로 그 외 19.1분보다 뚜렷이 짧음 — 도로가 한산할수록 같은 거리도 빨리 통과.

부스팅 + 타깃 인코딩으로 MAE 약 4분 (R^2 0.79)

튜닝한 히스토그램 부스팅·양상블이 test MAE 약 4.05분· R^2 0.794 달성. 평균 4분 안팎 오차로 소요시간을 예측.

양상블은 만능이 아님

세 모델을 가중 블렌딩해도 부스팅 단독 대비 개선은 미미(4.048→4.046분). 부스팅 0.77·랜덤포레스트 0.23(선형 0)로 부스팅이 지배 → 구성 모델 실력이 고를 때만 양상블 이득.

14. 제출 · 리더보드

최고 모델의 test 예측을 **submission.csv**(783,972행)로 저장. 모델 순위를 리더보드로 정리하고 예측값 표본을 함께 제시.

```
submission.csv — row_id · duration_min_pred 두 컬럼으로 test 전 행의 예측 소요시간 저장(python -m trip_duration.submit 로 재생성). 팀원과 동일한 test 셋을 쓰므로 이 예측을 실제값과 대조하면 순위 산정 가능.
```

14-1. 리더보드 — 모델 순위 (MAE 낮을수록 상위)

순위	모델	MAE (분)	RMSE (분)	R ²
1 ★	양상불 (블렌딩)	4.046	6.989	0.7944
2	히스토그램 부스팅 (튜닝)	4.048	7.005	0.7934
3	랜덤포레스트	4.205	7.180	0.7830
4	선형회귀 (Ridge)	5.403	8.980	0.6606

1위 · 양상불(블렌딩), test MAE 4.046분 · R² 0.794. 튜닝 부스팅이 근소차 2위(4.048분). 선형회귀 대비 오차를 약 1.4분 줄임 — 거리와 시각(혼잡)의 비선형 관계를 트리 계열이 잘 잡음.

14-2. 제출 예측 미리보기 — 실제 vs 예측 (표본 10건)

거리(mi)	시각	요일	실제(분)	예측(분)	오차(분)
0.7	15	금	6.4	7.9	1.5
2.2	1	토	18.4	14.3	4.1
8.0	15	토	21.4	28.0	6.7
1.6	19	월	11.0	9.2	1.8
0.5	12	수	12.8	11.6	1.3
1.9	17	일	16.5	15.2	1.3
1.7	18	화	14.4	14.2	0.2
6.6	16	수	34.4	30.7	3.7
1.3	14	일	5.5	10.2	4.7
3.6	23	금	17.3	21.9	4.6

읽는 법 · 각 행은 test 트립 하나. 짧은 도심 트립은 오차 1~2분으로 잘 맞히고, 장거리·비첨두 시간대처럼 변동이 큰 경우엔 오차가 다소 커짐 (표본 최대 6~7분). 전체 평균 오차(MAE)가 약 4분이라, '대략 몇 분 걸릴지'를 실용적으로 안내하는 수준.